Ethereum Virtual Machine

Ambient intelligence course (A.Y. 2025/2026)
Topic taught by Prof. Arnaldo Tomasino

Gabriele Colapinto

April 16, 2026

License

Notes from Ambient intelligence (A.Y. 2025/2026) by Gabriele Colapinto. Course taught by professors Michele Ruta, Floriano Scioscia, Arnaldo Tomasino, Davide Loconte — Licensed under CC BY-NC 4.0.
https://creativecommons.org/licenses/by-nc/4.0/

© 2025 Gabriele Colapinto
Released under the Creative Commons Attribution–NonCommercial 4.0 International License

Contents

1 The Ethereum Virtual Machine (EVM) 5
1.1 Core Concepts and Architecture . . . 5
1.1.1 State, Accounts, and Transactions . . . 5
1.1.2 The "Quasi-Turing-Complete" Nature of the EVM . . . 6
1.2 The Role and Functionality of Smart Contracts . . . 6
1.3 EVM Execution Environment . . . 6
1.3.1 Memory Areas: Stack, Memory, and Storage . . . 6
1.3.2 The EVM Instruction Set (Opcodes) . . . 6
1.4 The Gas Mechanism: Fueling the EVM . . . 9
1.5 The EVM Security Model . . . 10
1.6 Practical Example: EVM Bytecode Execution . . . 11
1.6.1 Example 1: Summing Two Values . . . 11
1.6.2 Example 2: Storing and Returning a Value . . . 12
1.7 Practical execution of the previous examples . . . 13
1.7.1 Environment setup . . . 13
1.7.2 Decompilation and debug of example 1 . . . 13
1.7.3 Decompilation and debug of example 2 . . . 14

1 The Ethereum Virtual Machine (EVM)

The Ethereum Virtual Machine (EVM) is the core computational engine of the Ethereum network. It is the environment where all smart contracts are deployed and all transactions are executed. This section deconstructs the EVM's architecture, execution model, and security principles, providing a foundational understanding of how decentralized applications function in a trustless environment.

1.1 Core Concepts and Architecture

The EVM can be defined in several ways, each highlighting a different aspect of its function:

- It is a **consensus-based, globally executed virtual machine**, meaning every full node in the network runs the EVM to validate transactions and maintain consensus.
- It is a **formal model of a virtual state machine** that specifies exactly how the Ethereum system state is altered by each transaction.
- It is the part of Ethereum that specifically **handles smart contract deployment and execution**.

From a practical perspective, the EVM can be conceptualized as a large, decentralized computer. However, its execution environment is fundamentally "lifeless," meaning the global state remains static until a transaction triggers a state transition.

1.1.1 State, Accounts, and Transactions

The Ethereum state is comprised of accounts. There are two distinct types of accounts:

- **Externally Owned Account (EOA):** This account is controlled by a private key held by a user. EOAs are used to initiate transactions, such as sending Ether or interacting with smart contracts.
- **Contract Account:** This account is controlled by its own code and possesses an internal, persistent storage. It is activated when it receives a transaction or a message from another contract.

When a transaction is sent from an EOA, its effect depends on the destination. If the destination is another EOA, the transaction may simply transfer Ether. If the destination is a contract account, the contract's code is activated and executed. This code can:

- Read the contents of the received message.
- Read and write to its own internal storage.
- Send messages to other contracts, triggering their execution in turn.

1.1.2 The "Quasi-Turing-Complete" Nature of the EVM

The EVM is described as "quasi-Turing-complete" rather than fully Turing-complete. This distinction is critical: while high-level smart contract languages like Solidity are designed to be Turing-complete (capable of expressing any computable function, including infinite loops), the EVM's execution environment imposes a hard constraint on all computation: **gas**. Every operation requires a fee, paid in gas, from a finite pool allocated by the transaction. This mechanism makes infinite loops prohibitively expensive and guarantees that all program executions eventually terminate, ensuring the EVM's deterministic nature.

1.2 The Role and Functionality of Smart Contracts

Smart contracts are the primary application unit on Ethereum. They are autonomous pieces of code that live on the blockchain and can serve several distinct purposes.

- **Data Store:** A contract can maintain a persistent data store, acting as a public database that is useful to other contracts or external applications.
- **Forwarder Contract:** A contract can function as an EOA with more complex access policies. For example, a multisignature wallet is a forwarder contract that only forwards a message after receiving approval from a required number of private keys.
- **Relationship Management:** Contracts can manage ongoing relationships between users. A common example is a bounty contract that automatically pays a reward to any user who submits a valid solution to a specified problem.
- **Software Library:** A contract can provide a set of functions that other contracts can call, serving as a reusable software library on the blockchain.

Contracts interact with each other by sending **messages**. These are similar to transactions but are generated by contracts rather than EOAs. Like transactions, messages that trigger computation also consume gas.

1.3 EVM Execution Environment

The EVM is a **stack-based virtual machine**. It operates on an instruction set of approximately 140 unique opcodes, each of which manipulates data within specific memory areas. Opcodes are **1 B** long (two hexadecimal digits) and each one of them has a **cost**.

1.3.1 Memory Areas: Stack, Memory, and Storage

A smart contract has access to three distinct memory areas during execution, each with unique characteristics and costs. These memory areas are listed in table 1.1.

1.3.2 The EVM Instruction Set (Opcodes)

EVM instructions, or opcodes, can be grouped into several categories: Stack manipulation, Memory/Storage manipulation, Arithmetic/comparison/bitwise, Program counter related, Environment related, and Halting.

Area	Characteristics	Purpose
Stack	Max size of 1024 items; each item is a 256-bit word.	Used by instructions to store operands and pass values to functions.
Working Memory	Linear array of bytes; volatile (cleared after execution).	Used for storing local function variables.
Storage	Persistent key-value store (256-bit keys and values); publicly readable; writing is very expensive compared to working memory.	Used to persist information between different smart contract calls.

Table 1.1: Memory areas with related characteristics and purpose

0x	0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
0	STOP	ADD	MUL	SUB	DIV	SDIV	MOD	SMOD	ADDMOD	MULMOD	EXP	SIGNEXTEND				
1	LT	GT	SLT	SGT	EQ	ISZERO	AND	OR	XOR	NOT	BYTE	SHL	SHR	SAR		
2	SHA3															
3	ADDRESS	BALANCE	ORIGIN	CALLER	CALLVALUE	CALLDATALOAD	CALLDATASIZE	CALLDATACOPY	CODESIZE	CODECOPY	GASPRICE	EXTCODESIZE	EXTCODECOPY	RETURNDATASIZE	RETURNDATACOPY	EXTCODEHASH
4	BLOCKHASH	COINBASE	TIMESTAMP	NUMBER	DIFFICULTY	GASLIMIT										
5	POP	MLOAD	MSTORE	MSTORE8	SLOAD	SSTORE	JUMP	JUMPI	PC	MSIZE	GAS	JUMPDIST				
6	PUSH1	PUSH2	PUSH3	PUSH4	PUSH5	PUSH6	PUSH7	PUSH8	PUSH9	PUSH10	PUSH11	PUSH12	PUSH13	PUSH14	PUSH15	PUSH16
7	PUSH17	PUSH18	PUSH19	PUSH20	PUSH21	PUSH22	PUSH23	PUSH24	PUSH25	PUSH26	PUSH27	PUSH28	PUSH29	PUSH30	PUSH31	PUSH32
8	DUP1	DUP2	DUP3	DUP4	DUP5	DUP6	DUP7	DUP8	DUP9	DUP10	DUP11	DUP12	DUP13	DUP14	DUP15	DUP16
9	SWAP1	SWAP2	SWAP3	SWAP4	SWAP5	SWAP6	SWAP7	SWAP8	SWAP9	SWAP10	SWAP11	SWAP12	SWAP13	SWAP14	SWAP15	SWAP16
a	LOG0	LOG1	LOG2	LOG3	LOG4											
b																
c																
d																
e																
f	CREATE	CALL	CALLCODE	RETURN	DELEGATECALL	CREATE2					STATICCALL			REVERT	INVALID	SELFDESTRUCT

Figure 1.1: EVM instruction set – The number of the row corresponds to the first hexadecimal digit of the opcode and the number of the column to its second digit. In fact, `0x60` $\longrightarrow$ `PUSH1`.

Table 1.2 shows a few examples of common opcodes and figure 1.1 shows a table associating opcodes to their mnemonic counterpart. The reader is encouraged to check the official EVM instruction set web page.

Stack Semantics and Memory Interaction

In the EVM, each instruction interacts with the stack in a well-defined way. This behavior can be formally described using two parameters:

- δ: the number of items removed (popped) from the stack
- α: the number of items added (pushed) onto the stack

Each opcode consumes δ elements from the top of the stack and produces α new elements, following a last-in, first-out (LIFO) paradigm.

Stack manipulation instructions These instructions directly modify the structure of the stack without performing computations.

Mnemonic	Opcode Category	Description
`POP`, `PUSHn`, `DUPn`, `SWAPn`	Stack Manipulation	Remove, insert, duplicate, or reorder elements on the stack.
`MLOAD`, `MSTORE`, `MSTORE8`, `MSIZE`	Memory Manipulation	Load from or store data into temporary memory; query memory size.
`SLOAD`, `SSTORE`	Storage Manipulation	Load from or store data into persistent contract storage.
`ADD`, `SUB`, `MUL`, `DIV`, `MOD`, `LT`, `GT`, `AND`, `OR`	Arithmetic / Comparison / Bitwise	Perform arithmetic operations, comparisons, and bitwise logic on stack values.
`JUMP`, `JUMPI` (jump if, conditional jump), `PC`, `JUMPDEST`	Program Counter Related	Modify or inspect the program counter and control execution flow.
`ADDRESS`, `CALLER`, `CALLVALUE`, `NUMBER`	Environment Related	Access information about the execution environment and transaction context.
`STOP`, `RETURN`, `REVERT`, `INVALID`, `SELFDESTRUCT`	Halting	Terminate execution, optionally returning data or reverting state changes.

Table 1.2: EVM opcodes grouped by category with representative examples

- `POP` ($\delta = 1, \alpha = 0$): removes the top element from the stack.
- `PUSHn` ($\delta = 0, \alpha = 1$): pushes an immediate value of n bytes onto the stack.
- `DUPn`: duplicates the n-th stack element and pushes it on top.
- `SWAPn`: swaps the top element with the n-th element below it.

Arithmetic instructions These instructions consume operands from the stack and push the result back.

- `ADD`, `SUB` ($\delta = 2, \alpha = 1$): pop two operands and push the result.

Memory interaction Some instructions interact with the EVM memory, which is a temporary, byte-addressable, and non-persistent data area.

- `MLOAD` ($\delta = 1, \alpha = 1$): pops a memory address from the stack, reads 32 bytes (one word) from memory at that address, and pushes the value onto the stack.
- `MSTORE` ($\delta = 2, \alpha = 0$): pops a memory address and a value from the stack, and stores the value (32 bytes) at the specified memory location.

Execution model The EVM is a stack-based machine: all computations are performed using stack operands. Memory is not accessed directly by instructions, but only indirectly through stack values that represent memory addresses.

For example, the instruction `MSTORE` does not contain explicit operands. Instead, it expects the required inputs (address and value) to be already present on the stack. This design makes the instruction set compact but requires careful tracking of stack state during execution.

Note on memory The EVM memory is sometimes informally compared to a heap, but it differs from traditional heap memory in conventional programming languages:

- it is linearly addressed and byte-oriented;
- it is reset for every transaction execution;
- it is not persistent (unlike storage);
- it automatically expands when accessed, incurring additional gas costs.

Therefore, instructions such as `MLOAD` and `MSTORE` provide the only interface between the stack and this temporary memory space.

1.4 The Gas Mechanism: Fueling the EVM

The gas mechanism is a critical feature of the EVM, designed to prevent Denial of Service (DoS) attacks and resource abuse by assigning a cost to every computational step. Every operation executed inside the EVM is performed by every full node in the network, as required by Ethereum's consensus model.

This design provides an important benefit: any smart contract can interact with any other contract without the overhead of external remote procedure calls. However, the drawback is that computation on the EVM is expensive, since it must be replicated across the entire network. As a rule of thumb, one should avoid performing on-chain computations that could be done more efficiently off-chain.

Every transaction must include two gas-related fields:

- `GASPRICE`: The fee (in wei, a small denomination of Ether) the sender is willing to pay per unit of gas. Transaction senders are incentivized to choose a reasonable `GASPRICE`, since values that are too low may lead validators (or miners, in legacy systems) to ignore the transaction.
- `STARTGAS`: The total amount of gas allocated for the transaction's execution.

When a transaction is processed, the sender must be able to afford the maximum possible cost. Therefore, an amount equal to `STARTGAS * GASPRICE`, together with the transferred value, is reserved (or effectively deducted) from the sender's account balance before execution begins.

The process of calculating the gas cost for a transaction is as follows:

- The initial gas available for computation is `STARTGAS` minus a base fee of 21,000 gas. An additional fee is subtracted based on the transaction's data size (for example, earlier versions of the protocol charged 68 gas per non-zero byte, later reduced to 16).

- For each instruction executed by the EVM, a specific amount of gas is subtracted from the remaining total, according to the cost of the corresponding opcode and possible dynamic factors such as memory expansion.
- If the execution completes successfully with `N >= 0` gas remaining, the sender is refunded an amount equal to `N * GASPRICE` in wei.
- If execution runs out of gas or fails, all state changes are reverted, but the gas consumed up to that point is not refunded.

The cost of most instructions falls into one of several groups listed in table 1.3.

Group	Gas Cost	Examples and Description
`Gzero`	0	`STOP`, `RETURN` — Instructions that terminate execution without additional computational cost.
`Gbase`	2	`ADDRESS`, `POP`, `PC` — Simple operations with minimal computational effort.
`Gverylow`	3	`ADD`, `SUB`, `LT`, `AND`, `MLOAD`, `MSTORE` — Basic arithmetic, logical, and memory operations.
`Glow`	5	`MUL`, `DIV`, `SDIV`, `MOD` — More expensive arithmetic operations.
`Gmid`	8	`ADDMOD`, `MULMOD`, `JUMP` — Medium-cost operations, including modular arithmetic and control flow changes.
`Ghigh`	10	`JUMPI` — Higher-cost conditional control flow operations.

Table 1.3: EVM instruction gas cost groups with representative examples

Note: The values reported in table 1.3 refer to the base cost of instructions. The actual gas cost may vary depending on the execution context, for example due to memory expansion, storage access (e.g., `SLOAD`, `SSTORE`), or other dynamic factors such as operands length (for example, the instruction `SHA3` costs 30 gas + 6 gas per word).

1.5 The EVM Security Model

The EVM is a security-oriented virtual machine designed to operate in a global, adversarial environment. Its security is enforced through four key restrictions:

1. **Paid Computation:** Every computational step must be paid for upfront through the gas mechanism. This prevents malicious actors from bogging down the network with resource-intensive, non-terminating programs.
2. **Sandboxed Execution:** A contract is executed in a sandbox. It may only access and modify its own internal state or trigger the execution of other contracts. It cannot affect anything else on the node's machine.
3. **Isolated State:** Contracts cannot directly access or read each other's state (storage). They may only interact by transmitting a single, arbitrary-length byte array as a message.

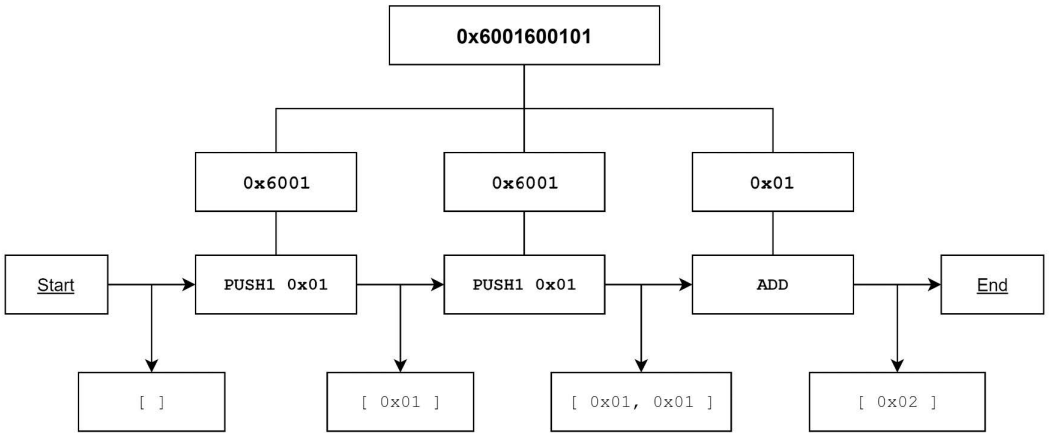

Figure 1.2: Bytecode summing the values 1 and 1 – The bottom row of blocks represents the stack.

4. **Deterministic Execution:** Program execution is fully deterministic. For any given starting state and transaction, every conforming EVM implementation will produce an identical final state. This is essential for achieving network-wide consensus.

1.6 Practical Example: EVM Bytecode Execution

To understand how these concepts come together, we can examine the execution of simple EVM bytecode.

1.6.1 Example 1: Summing Two Values

Consider the following bytecode, which adds the values 1 and 1.

- **Bytecode:** `0x6001600101`
- **Code visual inspection:** The code can be broken down by analyzing the bytecode hexadecimal string two digits at a time (see figure 1.2).
  1. `0x60 01` ⟶ `PUSH1 0x01`
  2. `0x60 01` ⟶ `PUSH1 0x01`
  3. `0x01` ⟶ `ADD`
- **Disassembled Code:**

```
00000: PUSH1 0x01
00002: PUSH1 0x01
00004: ADD
```

Note: The program counter (PC, the number before the colon character) does not increase by a fixed amount. Instead, it advances by the number of bytes of the current instruction. While most opcodes are 1 byte long, `PUSHn` instructions occupy `1 + n` bytes, since they include immediate

data.

The execution trace proceeds as follows:

1. The stack starts empty.
2. The first `PUSH1` instruction places the value `0x01` onto the stack. The stack is now `[0x01]`.
3. The second `PUSH1` instruction places another `0x01` onto the stack. The stack is now `[0x01, 0x01]`.
4. The `ADD` instruction pops the top two items (`0x01` and `0x01`), adds them, and pushes the result (`0x02`) back onto the stack. The final stack is `[0x02]`.

1.6.2 Example 2: Storing and Returning a Value

This example expands on the first by storing the result in memory and then returning it.

- **Bytecode:** `0x6001600101600052600160 1ff3`
- **Code visual inspection:**
  1. `0x60 01` ⟶ `PUSH1 0x01`
  2. `0x60 01` ⟶ `PUSH1 0x01`
  3. `0x01` ⟶ `ADD`
  4. `0x60 00` ⟶ `PUSH1 0x00`
  5. `0x52` ⟶ `MSTORE`
  6. `0x60 01` ⟶ `PUSH1 0x01`
  7. `0x60 1f` ⟶ `PUSH1 0x1f`
  8. `0xf3` ⟶ `RETURN`
- **Disassembled Code:**

```
00000: PUSH1 0x01
00002: PUSH1 0x01
00004: ADD
00005: PUSH1 0x00
00007: MSTORE
00008: PUSH1 0x01
0000a: PUSH1 0x1f
0000c: RETURN
```

The execution trace is more complex:

1. The first three instructions execute as before, leaving `[0x02]` on the stack.
2. `PUSH1 0x00` pushes the value `0x00` onto the stack, which now holds `[0x00, 0x02]`.
3. `MSTORE` pops the top two values. It uses `0x00` as the memory offset and `0x02` as the value to write. It stores `0x02` as a full 32-byte word at memory location 0.
4. `PUSH1 0x01` pushes `0x01` onto the stack. This will be the size of the data to return.
5. `PUSH1 0x1f` pushes `0x1f` (decimal 31) onto the stack. This will be the memory offset from which to read the return data. The stack now contains `[0x1f, 0x01]`.

6. `RETURN` pops the offset (`0x1f`) and size (`0x01`). It reads 1 byte of data starting from memory offset 31. Because the value `0x02` was stored as a 32-byte word, the byte at this position is `0x02`. This value is returned as the result of the execution.

1.7 Practical execution of the previous examples

1.7.1 Environment setup

To execute all the Ethereum operations on your machine you are highly encouraged to use a Linux distro inside a virtual machine in order to not interfere with your main host operating system. I use VirtualBox for the virtual machine generation and management and Ubuntu as the operating system of the virtual machine.

Inside the virtual machine, you need to install a C compiler (gcc is included in the **build-essential** package), the **Go language package** and **Ethereum Geth version 1.14** (the latest package does not have the same functionality required to run the examples).

```
sudo apt install build-essential golang-go
```

Listing 1.1: Bash command to install the Geth requirements

To install an older version of Ethereum Geth you can follow this guide. I installed the version **Geth & Tools 1.14.13**.

Once you installed everything, all you have to do is to create a folder to contain the two previously exposed examples.

1.7.2 Decompilation and debug of example 1

1. Save the bytecode as text readable file:

   ```
   echo 6001600101 > sum.bc
   ```

2. Disassembly the code with the disasm tool of the EVM:

   ```
   evm disasm sum.bc
   ```

   You will see the following result:

   ```
   6001600101
   00000: PUSH1 0x01
   00002: PUSH1 0x01
   00004: ADD
   ```

   The first row is the bytecode contained in the file and the rest of the output is the decompiled code we derived after the visual inspection.

3. Run the code in debug mode:

   ```
   evm --debug --code 6001600101 run
   ```

   You will see the following result:

```
* BLANK LINE *
#### TRACE ####
PUSH1 pc=00000000 gas=10000000000 cost=3

PUSH1 pc=00000002 gas=9999999997 cost=3
Stack:
00000000 0x1

ADD pc=00000004 gas=9999999994 cost=3
Stack:
00000000 0x1
00000001 0x1

STOP pc=00000005 gas=9999999991 cost=0
Stack:
00000000 0x2

#### LOGS ####
```

I have highlighted the fact that the first line is blank in the result because it's important to notice this detail. This is due to the fact that the considered program does not have a return statement so the output is null.

The output contains all the steps described in the previous section in which we described the code.
Note: The lines below an instruction represent the state of the stack at the time of the execution of the command. This is the reason why there isn't a stack description below the first command and the reason why the stack description below the second `PUSH1` command contains only the number pushed in the stack by the first command.

1.7.3 Decompilation and debug of example 2

1. Save the bytecode as text readable file:

```
echo 6001600101600052600160 1ff3 > sum2.bc
```

2. Disassembly the code with the disasm tool of the EVM:

```
evm disasm sum2.bc
```

You will see the following result:

```
6001600101600052600160 1ff3
00000: PUSH1 0x01
00002: PUSH1 0x01
00004: ADD
00005: PUSH1 0x00
00007: MSTORE
00008: PUSH1 0x01
0000a: PUSH1 0x1f
0000c: RETURN
```

The first row is the bytecode contained in the file and the rest of the output is the decompiled code we derived after the visual inspection.

3. Run the code in debug mode:

```
evm --debug --code 6001600101600052600160 1ff3 run
```

You will see the following result:

```
0x02
#### TRACE ####
PUSH1 pc=00000000 gas=10000000000 cost=3

PUSH1 pc=00000002 gas=9999999997 cost=3
Stack:
00000000 0x1

ADD pc=00000004 gas=9999999994 cost=3
Stack:
00000000 0x1
00000001 0x1

PUSH1 pc=00000005 gas=9999999991 cost=3
Stack:
00000000 0x2

MSTORE pc=00000007 gas=9999999988 cost=6
Stack:
00000000 0x0
00000001 0x2
Memory:
00000000 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00000010 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00

PUSH1 pc=00000008 gas=9999999982 cost=3
Memory:
00000000 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00000010 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 02

PUSH1 pc=00000010 gas=9999999979 cost=3
Stack:
00000000 0x1
Memory:
00000000 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00000010 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 02

RETURN pc=00000012 gas=9999999976 cost=0
Stack:
00000000 0x1f
00000001 0x1
Memory:
```

```
00000000 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00000010 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 02

#### LOGS ####
```

In this case, the program has a return statement so the first line of the output is the value returned by the program. Like in the previous example, the rest of the output contains all the steps described in the previous section in which we described the code.

`Note:` The output of the last command was edited to add the content of the contract memory. Unfortunately, it is not possible to show this information using more modern versions of Go and Gether. What really matters in this case is not the exact reproducibility of the experiment but understanding the process.

I invite you to observe the representation of the memory: EVM uses a big-endian logic meaning that the most significant bit has a lower address than the others. This is the reason why the value `0x02` is stored at the very end of the 32-byte word. The memory is represented using 16-bytes rows with a double space between the 7th and the 8th byte for a more tidy representation. $(00000010)_{16} = (16)_{10}$. For this reason the offset of the second row is `00000010`.

The Ethereum Virtual Machine, with its deterministic, gas-powered, and stack-based architecture, provides the secure and reliable foundation necessary for all smart contract functionality on the Ethereum network.